

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

stage01/
smartcare_v01.py
comparison.md
reflection.md
ai_usage.md

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored? **Appointment ID, Patient Name, Attending Doctor, Appointment Time**

What functions might be useful?

add_appointment(patient_name, attending_doctor, appointment_time)
view_appointments()
cancel_appointment(appointment_id)
search_appointments(query)
check_conflict(practitioner_name, appointment_time)

What could go wrong? **Double Booking, Invalid Time Formats, Accidentally booking in a past date, multiple patients with identical names**

What requirements are unclear? **Appointment Duration, Doctor availability, command line or GUI, is data saved permanently**

Part B - Build a Human-Written Prototype: AI OFF

#task 1

```
# Create and run a simple Python file with basic input,output statements

print("Welcome to SmartCare: Community Clinic Appointment Booking System!")

# First Appointment
patient1_name = 'Alice Smith'
practitioner1_name = 'Dr. John Doe'
appointment1_time = '2024-07-20 10:00 AM'
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time:
{appointment1_time}")

# Second Appointment
patient2_name = 'Bob Johnson'
practitioner2_name = 'Dr. Jane Roe'
appointment2_time = '2024-07-20 11:30 AM'
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time:
{appointment2_time}")
```

#task1enhanced

```
# Use lists, dictionaries and functions to enhance the Python file

appointments = []

def book_appointment(patient_name, practitioner_name, appointment_time):
    if not patient_name:
        raise ValueError("Patient name cannot be empty")
    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }
    appointments.append(appointment)

def display_appointments():
    if not appointments:
        print("No appointments recorded.")
        return
    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()
```

^ Now, run both programs , and identify at least five limitations.

No input validation
No Date/Time formatting check
No cancel appointment function
No edit appointment function
No preventing double-booking

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Variable/function/method naming not as good as CoPilot	CoPilot's naming more intuitive
Runs successfully?	Yes (About the same as CoPilot)	Yes
Uses only required features?	Yes (About the same as CoPilot)	Yes
Adds assumptions?	Yes (About the same as CoPilot)	Yes
Handles errors?	No exception handling as it hasn't been taught yet	No (lab says to keep it beginner-friendly when prompting, so it didn't do handling)
Could I explain it?	Yes, definitely	Yes, more intuitive than mine

Part F - Verify Behaviour

- Normal appointment
- Blank patient name
- Two appointments for the same practitioner/time
- Strange input such as patient_name=None or appointment_time=None

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

Part H - Reflection (150-250 words)

What did you build before using AI?

What did AI help you understand?

Did AI make assumptions?

How did you verify the AI output?

What engineering work remained for you?

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.